BÁO CÁO GIỮA KỲ • NHẬP MÔN HỆ THỐNG NHÚNG

XE TỰ HÀNH NÉ VẬT CẢN

SINH VIÊN THỰC HIỆN

24022889 - Đỗ Minh

24022909 - Nguyễn Văn Thắng

24022919 - Phạm Thành Trung

BÀI TOÁN, MỤC TIÊU VÀ PHẠM VI

Phân tích bài toán, xác định các thách thức điều hướng và thiết lập chỉ tiêu thi đấu

I

Đặt vấn đề & Thách thức

Mô hình thi đấu:
Xe robot tự hành (Micromouse) thi đấu trên sa hình tiêu chuẩn có sẵn.

Thách thức điều hướng:
• Điều hướng chính xác qua các khúc cua chữ U.
• Né các vật cản tĩnh lẫn vật cản động xuất hiện ngẫu nhiên trong thời gian giới hạn.

Ràng buộc thi đấu:
Đảm bảo xe luôn bám sát vạch, không để bánh xe ra ngoài biên hoặc đi tắt qua vùng cấm.

II

Mục tiêu trọng tâm

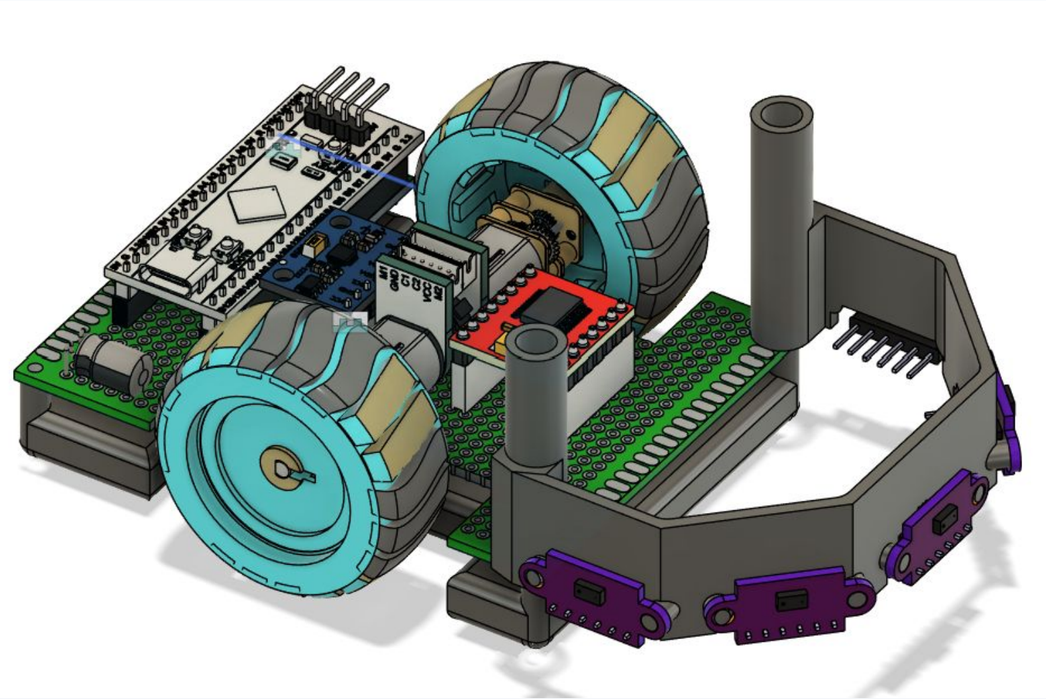
1. Nhận diện và phản xạ thời gian thực:
Cảm biến nhận diện vật cản theo thời gian thực với độ trễ tối thiểu, đảm bảo phát hiện tức thì vật cản bất ngờ.

2. Tối ưu hóa thuật toán điều khiển:
Tối ưu thuật toán dò đường và né vật cản để đạt tốc độ cao kết hợp độ chính xác tuyệt đối.

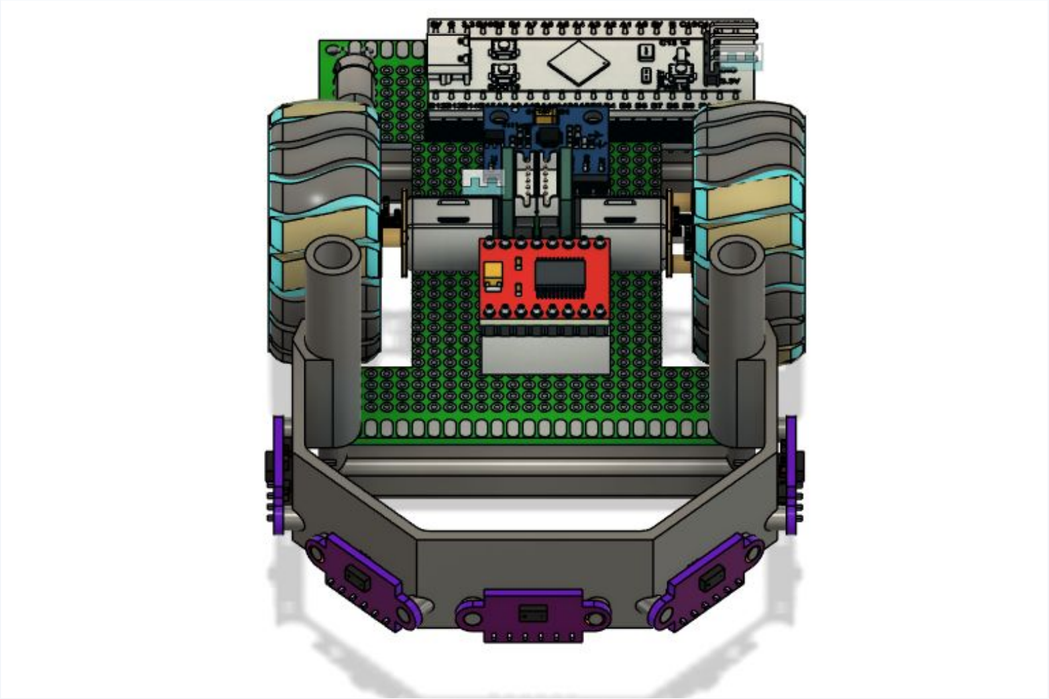
3. Hiệu suất thi đấu vượt trội:
Về đích an toàn, tối ưu thời gian di chuyển toàn trình và chạy sạch hoàn toàn không phạm lỗi.

Thiết kế mô hình 3D xe robot

Mô hình cơ khí nhỏ gọn, trọng tâm thấp, bố trí cụm cảm biến vòng cung.



Phối cảnh góc nghiêng: Bố trí cụm cảm biến cung tròn & bánh xe

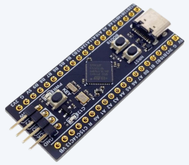


Hình chiếu trên xuống: Phân bổ trọng tâm mạch điều khiển & nguồn

Danh sách linh kiện phần cứng

Hệ thống vi điều khiển ARM Cortex-M4, động cơ DC Servo, driver công suất và cảm biến

1. **Vi điều khiển chính:** STM32F411CEU6 (Blackpill) - ARM Cortex-M4 100MHz
2. **Động cơ truyền động:** DC Servo GA12 N20 300RPM kèm encoder
3. **Mạch cầu công suất:** Driver motor TB6612FNG (Dual H-Bridge)
4. **Cảm biến khoảng cách:** Cảm biến quang ToF VL53L0X (I2C)
5. **Cảm biến góc nghiêng:** Cảm biến IMU 6 trục MPU6050
6. **Mạch nguồn ổn áp:** Mạch hạ áp xung DC-DC CN3903
7. **Mạch nạp / Gỡ lỗi:** Mạch nạp ST-Link V2 hỗ trợ SWD
8. **Module bluetooth:** Mạch Bluetooth HC-05 cấu hình UART
9. **Nguồn:** Pin LiPo 7.4V 1200mAh



STM32F411



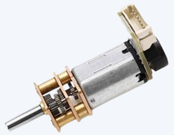
HC-05



ST-Link



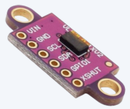
TB6612FNG



N20 Motor



MPU6050



VL53L0X



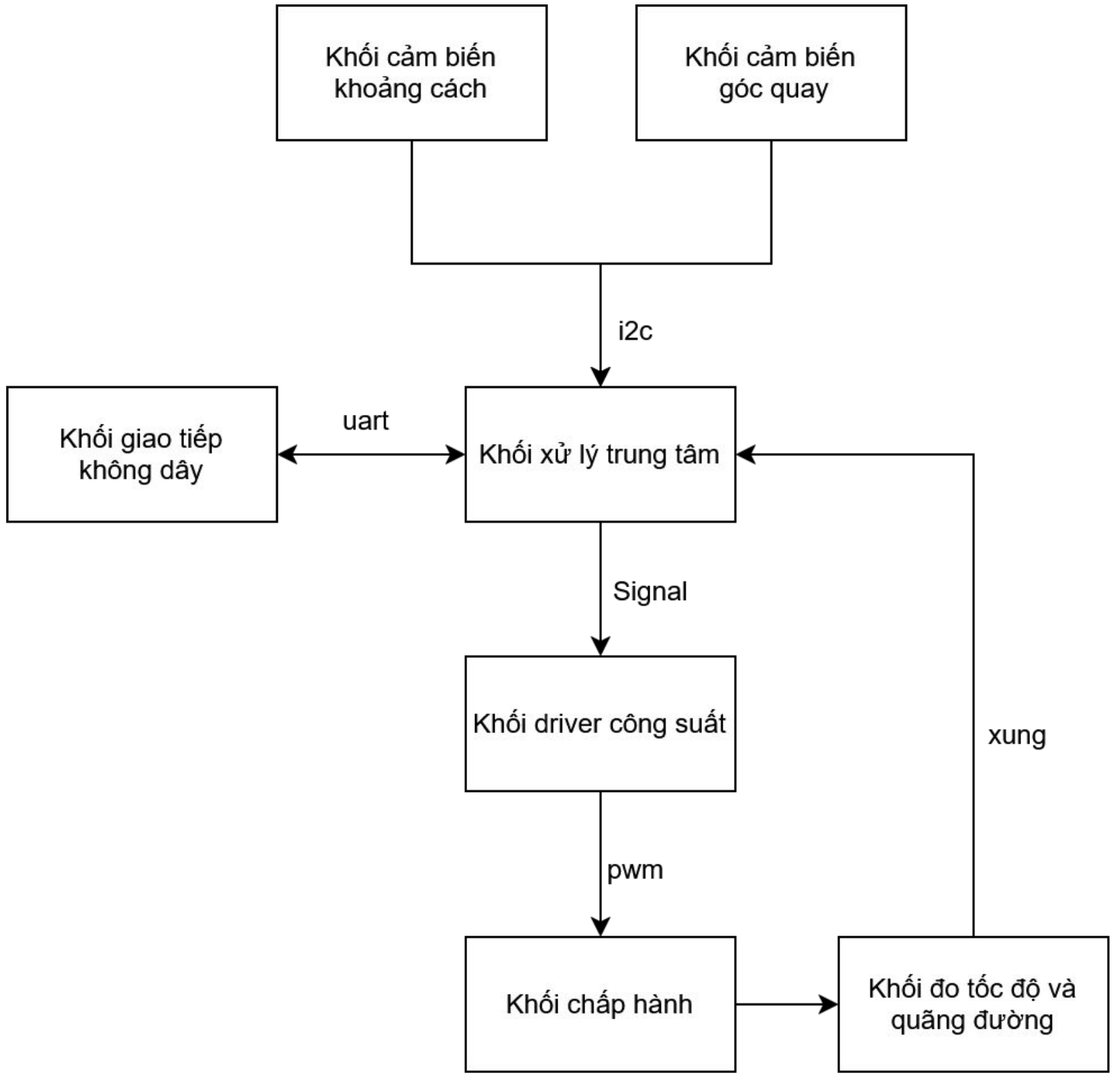
CN3903



Pin LiPo

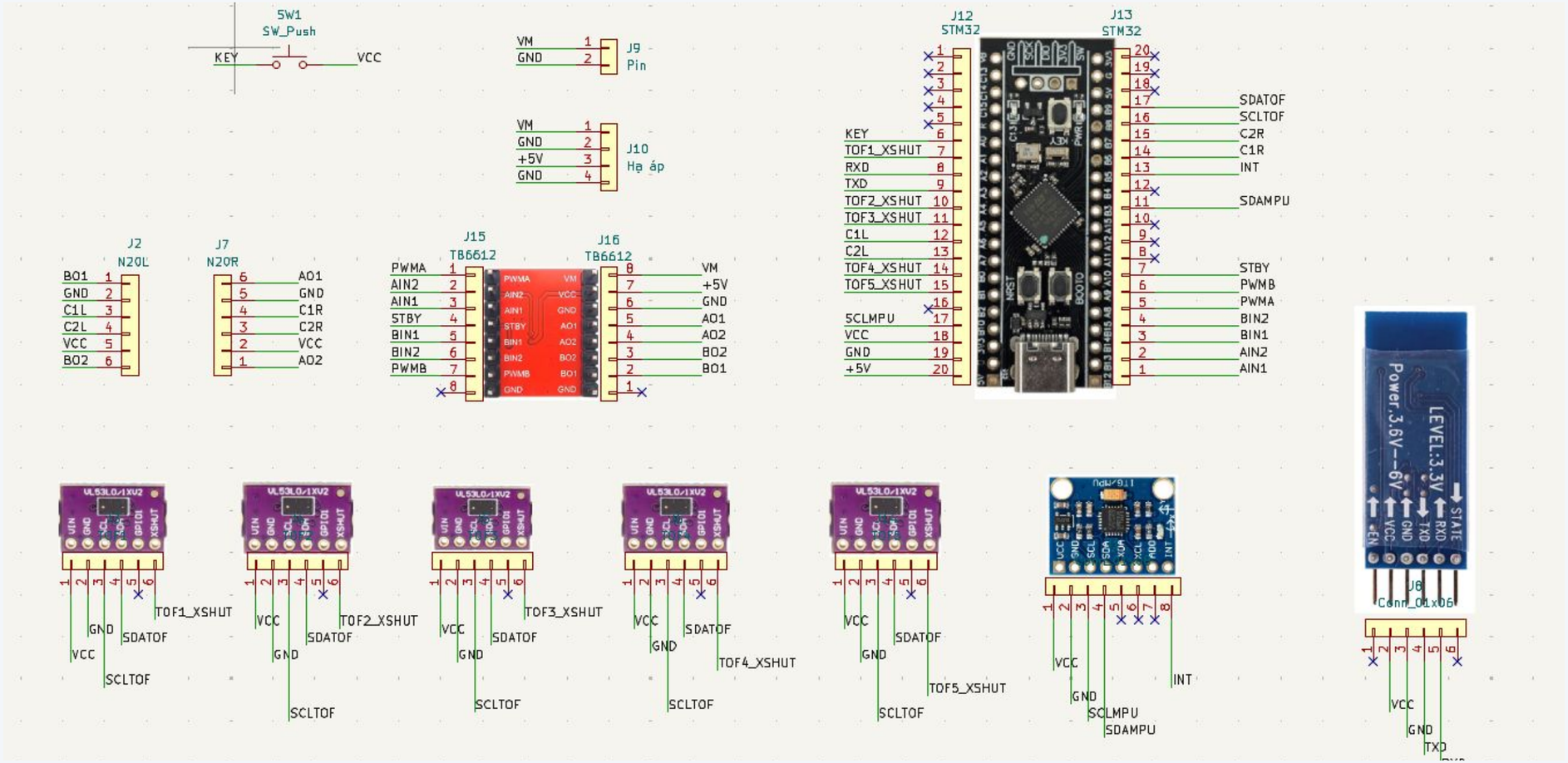
Sơ đồ luồng xử lý toàn hệ thống

Luồng dữ liệu khép kín: Cảm biến thu nhận → Vi điều khiển trung tâm → Điều khiển công suất & Động cơ



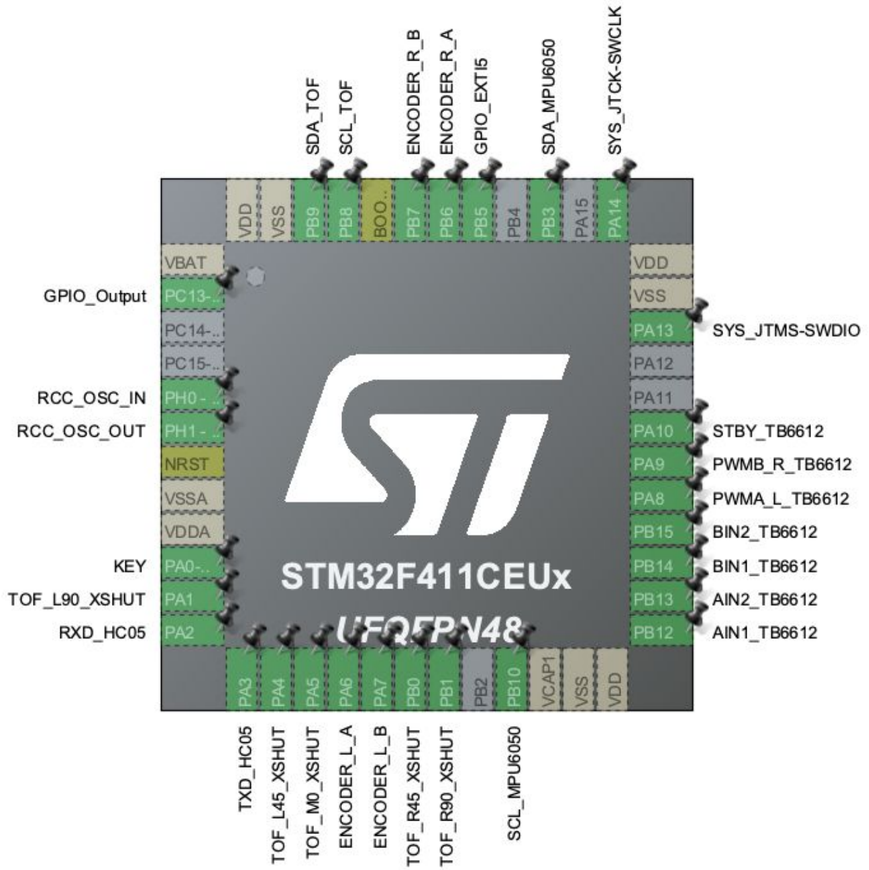
Sơ đồ kết nối mạch phần cứng

Chi tiết kết nối bus I2C cho mảng 5 cảm biến ToF, giao tiếp UART, nguồn và mạch cầu motor



Quy hoạch chân vi điều khiển STM32F411

Phân bổ GPIO cho PWM timer điều khiển motor, bus I2C phần cứng cho cụm cảm biến và chân SWD nạp firmware



Lập quỹ đạo tối ưu trước khi thi

Chạy trên máy tính: sân và 17 box cố định đã biết → tính sẵn đường ngắn, an toàn rồi nạp vào xe

1 **Bản đồ → trường khoảng cách (SDF)**

Lưới 5 mm: tra nhanh khoảng cách tới box và biên.
Xe 15 × 10 cm mô hình bằng các điểm trên chu vi.

2 **Hybrid A* trong không gian (x, y, θ)**

Chỉ sinh đường xe đi được: bán kính quay ≥ 150 mm.
Dijkstra 2D, thử nhiều điểm xuất phát.

3 **Làm mượt bằng tối ưu L-BFGS**

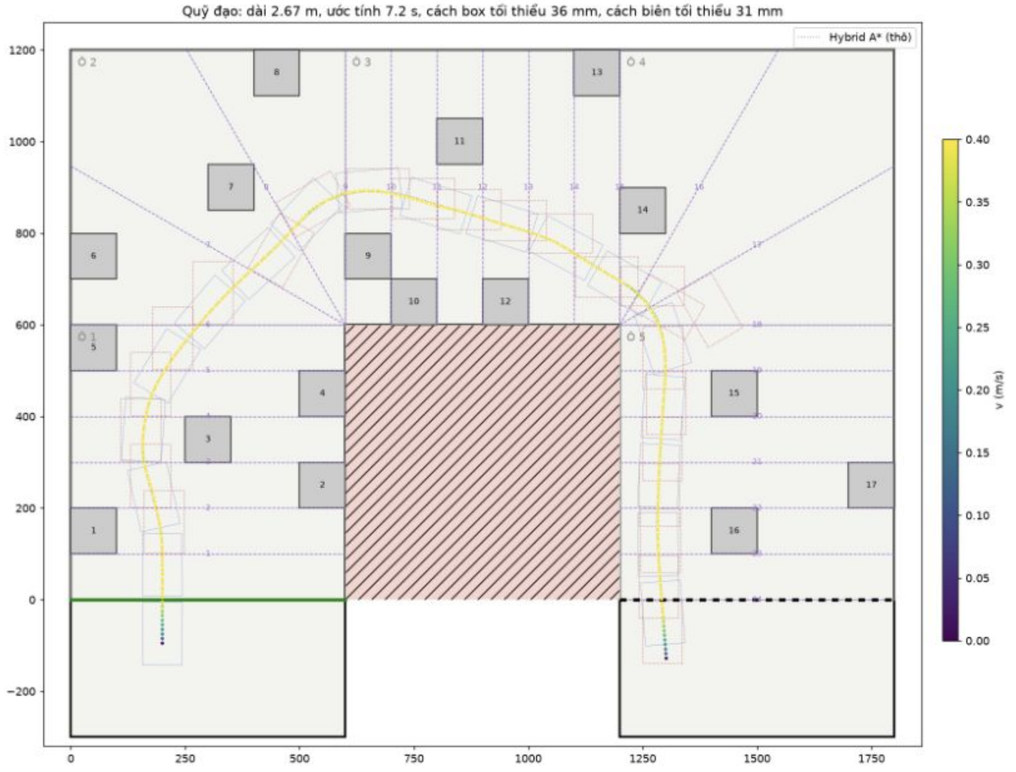
Tối ưu đa mục tiêu: cân bằng độ mượt, độ cong,
bám quỹ đạo gốc và giữ khoảng cách an toàn với vật cản.

4 **Spline bậc 3 + hồ sơ vận tốc**

Nội suy đường cong liên tục, giới hạn gia tốc ngang
ở khúc cua và quy hoạch hồ sơ vận tốc tăng/giảm êm.

5 **Xuất trajectory_data.h cho firmware**

Bảng điểm (s, x, y, θ, κ, v), 24 vạch chia phần
và vị trí đặt lại xe khi bị reset.



2.67 m
quãng đường

≈ 7.2 s
thời gian ước tính

≥ 36 mm
cách box gần nhất

24/24
vạch chia phần

Hybrid A*: tìm đường cho xe có ràng buộc chuyển động

Vì **A* lưới** tạo đường gấp khúc (45°/90°) không phù hợp động học xe, còn **Hybrid A*** tạo đường cong trơn xe thực tế bám được.

1 Trạng thái liên tục, lưới chỉ để gộp node

Node lưu tư thế liên tục (x, y, θ), không làm tròn.
Lưới 20 mm × 5° chỉ để gộp: mỗi ô giữ 1 node.

2 Mở rộng bằng cung chuyển động

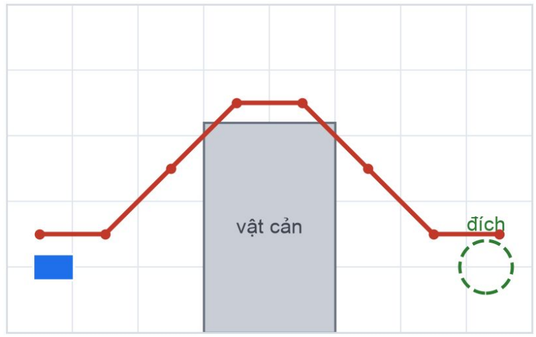
Mở rộng cây tìm kiếm bằng các cung chuyển động thực tế, tuân thủ giới hạn góc lái và bán kính quay xe.

- **Mô phỏng động học:** Sinh các nhánh thử nghiệm (đi thẳng, rẽ nông, rẽ sâu).
- **Kiểm tra va chạm:** Tra cứu trường khoảng cách (SDF) tại các điểm trên chu vi xe.

3 Hàm đánh giá chi phí (f = g + ε·h)

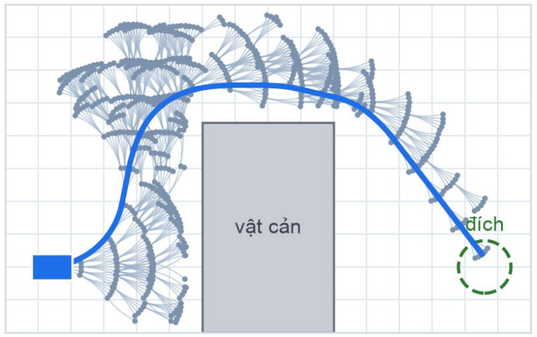
- **Chi phí thực tế g:** Tính theo quãng đường đã đi, phạt thêm độ cong và phạt chuyển hướng.
- **Chi phí ước lượng h:** Dijkstra 2D ngược từ đích (bỏ ràng buộc quay, đảm bảo an toàn).
- **Trọng số tăng tốc ε:** Hệ số ε ≥ 1 giúp định hướng tìm kiếm đường đi nhanh chóng hơn.

A* lưới



Không dùng: Đi qua tâm ô, góc gãy 45° / 90° (xe không bám được)

Hybrid A*



Dùng Hybrid A*: Cung chuyển động → Đường cong trơn (tuân thủ bán kính quay)

Hạn chế: gộp node theo ô nên không tối ưu tuyệt đối, đường còn gãy nhẹ → làm mượt L-BFGS.

Khi bị kẹt: thêm cung lùi (phạt ×w_rev, phạt mỗi lần đổi chiều) và xoay tại chỗ 30°.

Bám quỹ đạo và né box ngẫu nhiên

Chạy trên xe, chu kỳ 1 kHz: encoder + gyro + 5 ToF → định vị → phát hiện → lập lại đường → điều khiển

1 **Định vị bằng EKF**

Encoder + gyro dự đoán; ToF đo box đã biết để hiệu chỉnh.

2 **Phát hiện box ngẫu nhiên**

ToF ngắn hơn dự đoán < 50 mm → có vật lạ.
Ước lượng tâm box liên tục, phình theo độ bất định.

3 **Lập lại đường đi**

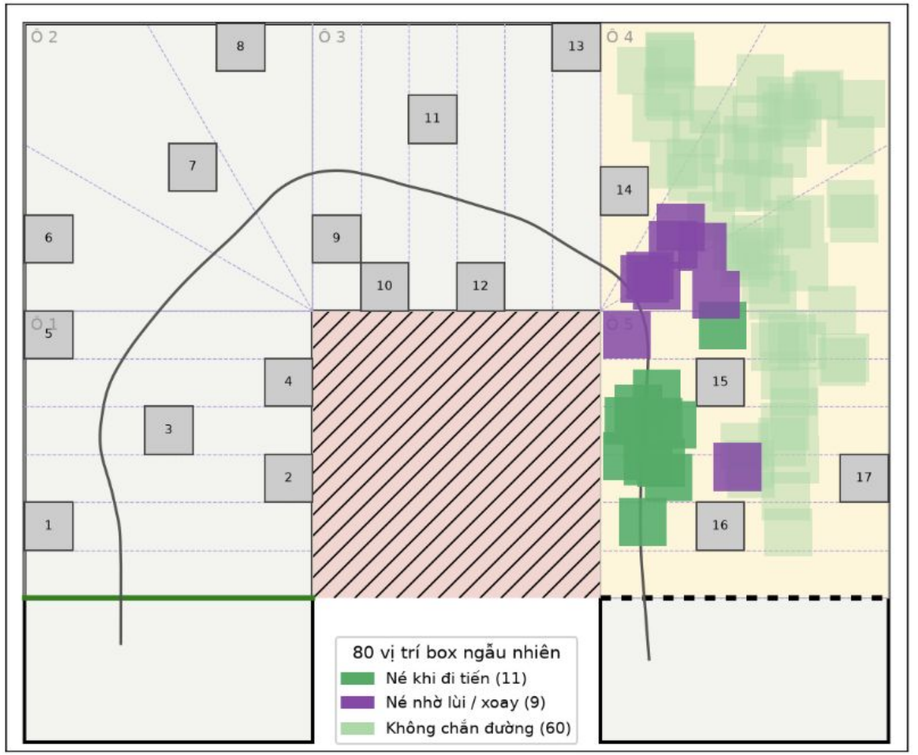
A* có trọng số rẽ nhánh rồi nhập lại quỹ đạo chính;
bị kẹt → Hybrid A* cho phép lùi và xoay tại chỗ.

4 **Bám quỹ đạo: bộ điều khiển Kanayama**

Tự động tính tốc độ chuẩn theo độ cong, đồng thời sửa lệch hướng và kéo xe về đúng đường đi.

5 **An toàn**

ToF trước báo quá gần 2 lần liên tiếp → dừng khẩn;
đường thông → tự chạy tiếp.



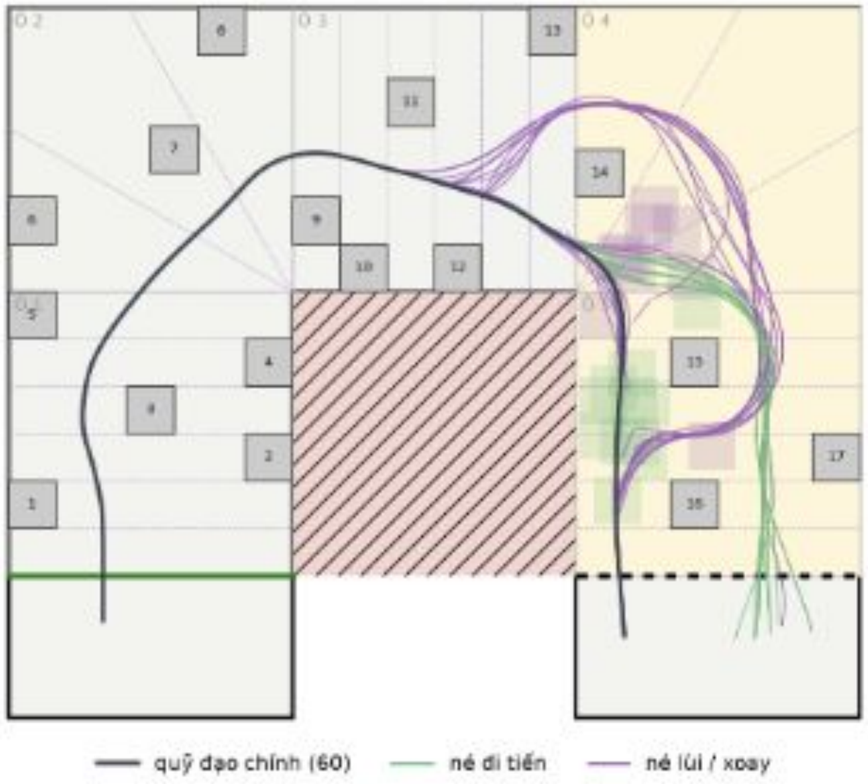
30/30	80/80	0	19
chạy sạch (nhiễu ×1.5)	vị trí về đích	lần bị trừ điểm	lần né (8 lùi / xoay)

Kết quả MÔ PHỎNG Python: có trượt bánh, sai số encoder / gyro, nhiễu và số đo đột biến của ToF. Chưa kiểm chứng trên xe thật.

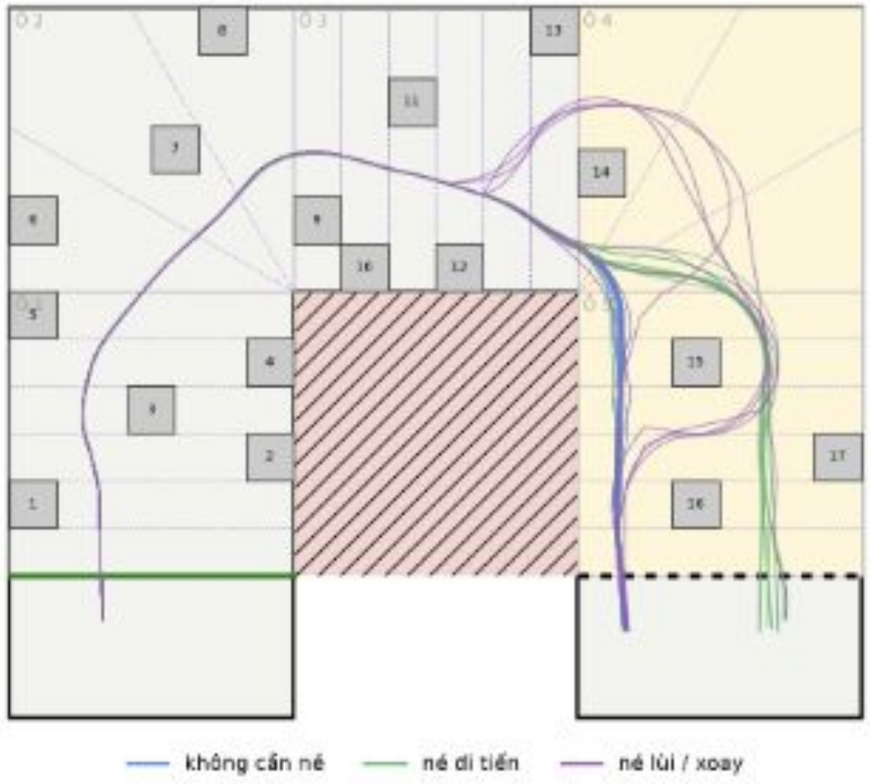
Thuật toán luôn sinh quỹ đạo hợp lệ

80 vị trí box ngẫu nhiên → 80 quỹ đạo do thuật toán lập, chồng lên cùng một bản đồ

Quỹ đạo do thuật toán lập



Đường xe chạy mô phỏng



80/80
quỹ đạo lập thành công

≥ 26 mm
cách box (giới hạn 25 mm)

≥ 3 mm
thân xe cách biên sân

≤ 33 mm
lệch quỹ đạo khi chạy

0
va chạm / bánh ra vạch

Mỗi vị trí box: thuật toán lập lại đường (Hybrid A* + làm mượt), rồi mô phỏng xe chạy có nhiễu. Về đích 80/80, bị trừ điểm 0 lần. Kết quả MÔ PHỎNG.

Kiến trúc firmware & kết quả test phần cứng

Driver dùng chung cho firmware test và firmware thi đấu; từng linh kiện được kiểm chứng trên xe thật qua bảng điều khiển web

1 Kiến trúc firmware phân lớp

Ứng dụng

- **hwtest** — test qua web (UART)
- **main** — thi đấu · đang phát triển

Bánh xe

- **pid** — bù trước động cơ · PID vị trí · profile hình thang

Driver

- **tof** — 5 ToF
- **mpu** — MPU6050
- **encoder** — TIM3/4
- **tb6612** — động cơ

HAL

- **STM32Cube HAL** — I2C1 · I2C2 · TIM1/3/4 · USART2 DMA · nhịp 1 kHz

2 Kết quả test trên xe thật

Bus I2C	OK	Đủ 5 ToF (0x30–0x34) + MPU (0x68), ổn định qua quét lặp
ToF VL53L0X	4/5	4 cảm biến ổn định, sai số 1–3 cm; R45 lỗi phần cứng, đang thay
MPU6050	OK	Nhiễu gyro ≈ 0,07°/s sau hiệu chuẩn; tự bù trôi khi đứng yên
Encoder	OK	1400 xung / vòng bánh, khớp lý thuyết 7 × 4 × 50; chu vi 140 mm
Động cơ + PID	OK	Tiến, lùi, quay ổn định; PID kín vòng khắc phục vượt đích của chạy hở
Quay tại chỗ	OK	Kết hợp MPU + encoder: tới đúng góc đặt, sai số góc rất nhỏ
UART / HC-05	OK	38400 baud qua USB-TTL và Bluetooth, telemetry 25 Hz

CẢM ƠN THẦY & CÁC BẠN ĐÃ LẮNG NGHE